Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №9
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Поиск расстояний в графе»

Выполнил: ст. гр. 25ВВВ1
Какулин В.С.
Самсонов М.А.
Проверил: к.т.н. доцент
Юрова О.В.
к.т.н., ст. преподователь
Деев М.В.

Пенза 2026

Название:

Поиск расстояний в графе.

Цель работы:

Изучить способы поиска расстояний в неориентированном графе на основе обхода в ширину (BFS) и обхода в глубину (DFS), реализовать их для матричной и списочной форм представления графа и сравнить время работы реализаций для графов разных порядков.

Лабораторное задание:

Задание 1:

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс queue из стандартной библиотеки C++.

3. Реализуйте процедуру поиска расстояний для графа, представленного списками смежности.

Задание 2:

1. Реализуйте процедуру поиска расстояний на основе обхода в глубину.

2. Реализуйте процедуру поиска расстояний на основе обхода в глубину для графа, представленного списками смежности.

3. Оцените время работы реализаций алгоритмов поиска расстояний на основе обхода в глубину и обхода в ширину для графов разных порядков.

Ход работы:

Описание используемых алгоритмов:

Расстоянием от исходной вершины до вершины графа называется наименьшее количество рёбер, по которым необходимо пройти, чтобы достичь этой вершины. Для поиска расстояний используются процедуры обхода графа, в которых вектор посещённых вершин заменяется вектором расстояний DIST. Непосещённые вершины помечаются значением −1, так как значения 0 и 1 сами могут быть расстояниями; расстояние от исходной вершины до самой себя равно 0.

При обходе в ширину вершины просматриваются по уровням удалённости: сначала все соседи исходной вершины (расстояние 1), затем их непосещённые соседи (расстояние 2) и т. д. Поэтому в момент первого попадания в вершину её расстояние уже минимально и вычисляется как DIST[i] = DIST[v] + 1, где v - текущая вершина.

При обходе в глубину алгоритм уходит по одному пути как можно дальше, поэтому вершина может быть впервые достигнута не по кратчайшему пути. Чтобы получить правильные расстояния, вершина посещается повторно каждый раз, когда к ней найден более короткий путь (DIST[i] > DIST[v] + 1), и её расстояние уточняется.

- **bfs_distance_matrix(start, size, G, DIST, print_order)**: поиск расстояний на основе обхода в ширину по матрице смежности с использованием класса queue. Помещает исходную вершину в очередь и устанавливает DIST[start] = 0. Пока очередь не пуста, извлекает первую вершину v, выводит её номер и просматривает строку G[v]: каждая смежная вершина i с DIST[i] = −1 помещается в очередь, и для неё устанавливается DIST[i] = DIST[v] + 1.
- **bfs_distance_lists(start, size, adjLists, DIST, print_order)**: аналогичная процедура для графа, представленного списками смежности: вместо строки матрицы перебирается очередь соседей текущей вершины.
- **dfs_distance_matrix(v, d, size, G, DIST)**: рекурсивный поиск расстояний на основе обхода в глубину по матрице смежности. Записывает в DIST[v] расстояние d, затем для каждой смежной вершины i, которая ещё не посещена или до которой найден более короткий путь (DIST[i] > d + 1), рекурсивно вызывает себя с расстоянием d + 1. Первый вызов выполняется для исходной вершины с d = 0.
- **dfs_distance_lists(v, d, adjLists, DIST)**: аналогичная процедура для графа, представленного списками смежности.

Результат выполнения работы:

```
Введите количество вершин графа: 10
Введите вероятность появления ребра в процентах (0-100): 30

Матрица смежности G (10 вершин, вероятность ребра - 30%):
    v0  v1  v2  v3  v4  v5  v6  v7  v8  v9
v0  0   1   1   1   1   1   0   0   0   1
v1  1   0   1   0   0   0   0   0   1   0
v2  1   1   0   1   0   0   0   1   0   1
v3  1   0   1   0   1   0   0   0   1   0
v4  1   0   0   1   0   0   0   0   1   1
v5  1   0   0   0   0   0   0   0   1   1
v6  0   0   0   0   0   0   0   1   0   0
v7  0   0   1   0   0   0   1   0   0   0
v8  0   1   0   1   1   1   0   0   0   0
v9  1   0   1   0   1   1   0   0   0   0

Списки смежности:
v0 : 1 2 3 4 5 9
v1 : 0 2 8
v2 : 0 1 3 7 9
v3 : 0 2 4 8
v4 : 0 3 8 9
v5 : 0 8 9
v6 : 7
v7 : 2 6
v8 : 1 3 4 5
v9 : 0 2 4 5

Введите начальную вершину (от 0 до 9): 0

Обход в ширину (BFS), граф задан МАТРИЦЕЙ смежности
Порядок обхода вершин: 0 1 2 3 4 5 9 8 7 6
Расстояния от вершины 0:
  до v0: 0
  до v1: 1
  до v2: 1
  до v3: 1
  до v4: 1
  до v5: 1
  до v6: 3
  до v7: 2
  до v8: 2
  до v9: 1
Время: 0.000004 сек.

Обход в ширину (BFS), граф задан СПИСКАМИ смежности
Порядок обхода вершин: 0 1 2 3 4 5 9 8 7 6
Расстояния от вершины 0:
  до v0: 0
  до v1: 1
  до v2: 1
  до v3: 1
  до v4: 1
  до v5: 1
  до v6: 3
  до v7: 2
  до v8: 2
  до v9: 1
Время: 0.000007 сек.
```

Рисунок 1 - результат выполнения программы при количестве вершин равным 10 (поиск расстояний обходом в ширину)

```
Обход в глубину (DFS), граф задан МАТРИЦЕЙ смежности
Расстояния от вершины 0:
  до v0: 0
  до v1: 1
  до v2: 1
  до v3: 1
  до v4: 1
  до v5: 1
  до v6: 3
  до v7: 2
  до v8: 2
  до v9: 1
Время: 0.000003 сек.

Обход в глубину (DFS), граф задан СПИСКАМИ смежности
Расстояния от вершины 0:
  до v0: 0
  до v1: 1
  до v2: 1
  до v3: 1
  до v4: 1
  до v5: 1
  до v6: 3
  до v7: 2
  до v8: 2
  до v9: 1
Время: 0.000017 сек.
```

Рисунок 2 – результат выполнения программы при количестве вершин равным 10 (поиск расстояний обходом в глубину)

Оценка времени работы алгоритмов поиска расстояний:

Таблица 1. Время поиска расстояний (N – количество вершин, p – вероятность появления ребра)

N	p, %	BFS, матрица (сек.)	BFS, списки (сек.)	DFS, матрица (сек.)	DFS, списки (сек.)
250	1	0.000137	0.000103	0.002973	0.002368
250	10	0.000243	0.000275	0.022714	0.032171
250	50	0.000462	0.000996	0.057981	0.124963
500	1	0.000614	0.000305	0.086018	0.039640
500	10	0.000925	0.000985	0.202038	0.223059
500	50	0.001859	0.003805	0.499061	0.964099
1000	1	0.002452	0.000746	0.973627	0.273617
1000	10	0.003477	0.003375	1.686327	1.651799
1000	50	0.007994	0.015790	4.090263	8.085782
2000	1	0.010249	0.002277	9.358789	1.850949
2000	10	0.014095	0.013040	14.239536	13.751228
2000	50	0.030023	0.059480	33.169457	63.706445

Время обхода в ширину по матрице смежности растёт как $O(N^2)$, так как для каждой вершины просматривается вся строка матрицы: при увеличении N с 1000 до 2000 время выросло примерно в 4 раза.

Обход в ширину по спискам смежности имеет сложность O(N + E), где E - число рёбер, поэтому на разреженном графе (p = 1%) списки быстрее матрицы примерно в 4,5 раза. На плотном графе (p = 50%) списки медленнее примерно в 2 раза из-за копирования очереди соседей каждой вершины.

Обход в глубину работает на несколько порядков медленнее обхода в ширину (при N = 2000 и p = 50% - 33,2 сек. против 0,03 сек.). При обходе в ширину каждая вершина обрабатывается один раз, а при обходе в глубину вершина сначала достигается по длинному пути, и затем её расстояние многократно уточняется. Представление графа влияет на обход в глубину так же: на разреженном графе списки быстрее матрицы, на плотном - медленнее.

Таким образом, для поиска расстояний в графе лучше использовать обход в ширину, а списки смежности эффективнее матрицы только на разреженных графах.

Вывод:

В ходе выполнения лабораторной работы были изучены и реализованы алгоритмы поиска расстояний в неориентированном графе на основе обхода в ширину и обхода в глубину для матричной и списочной форм представления графа. Все реализации дают одинаковые расстояния и корректно обрабатывают недостижимые вершины. Замеры времени показали, что обход в ширину решает задачу значительно быстрее обхода в глубину, а списки смежности выигрывают у матрицы только на разреженных графах.